

TECHNICAL PRESENTATION

DATA PREPROCESSING AND VISUALIZATION

Using Python Pandas

DSA0404: Fundamentals of data science

NAME: Sania Herbert

Reg no: 192324062

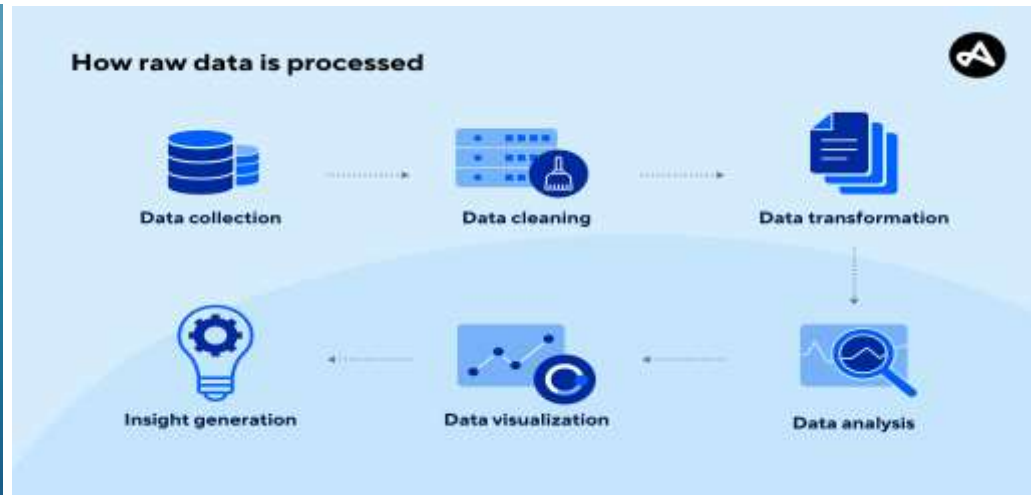
GUIDED BY: DR. Krishnaraj M

Data Preprocessing

Data preprocessing is the process of converting raw, incomplete, inconsistent, or noisy data into a clean and meaningful format before analysis or machine learning.

Real-Life Examples

- Hospital patient records often contain missing ages, duplicate patient IDs, or incorrect values. Data preprocessing cleans these records before analysis.



Why Data Preprocessing Matters

"80% of data science time is spent on data cleaning and preparation" - Industry Standard

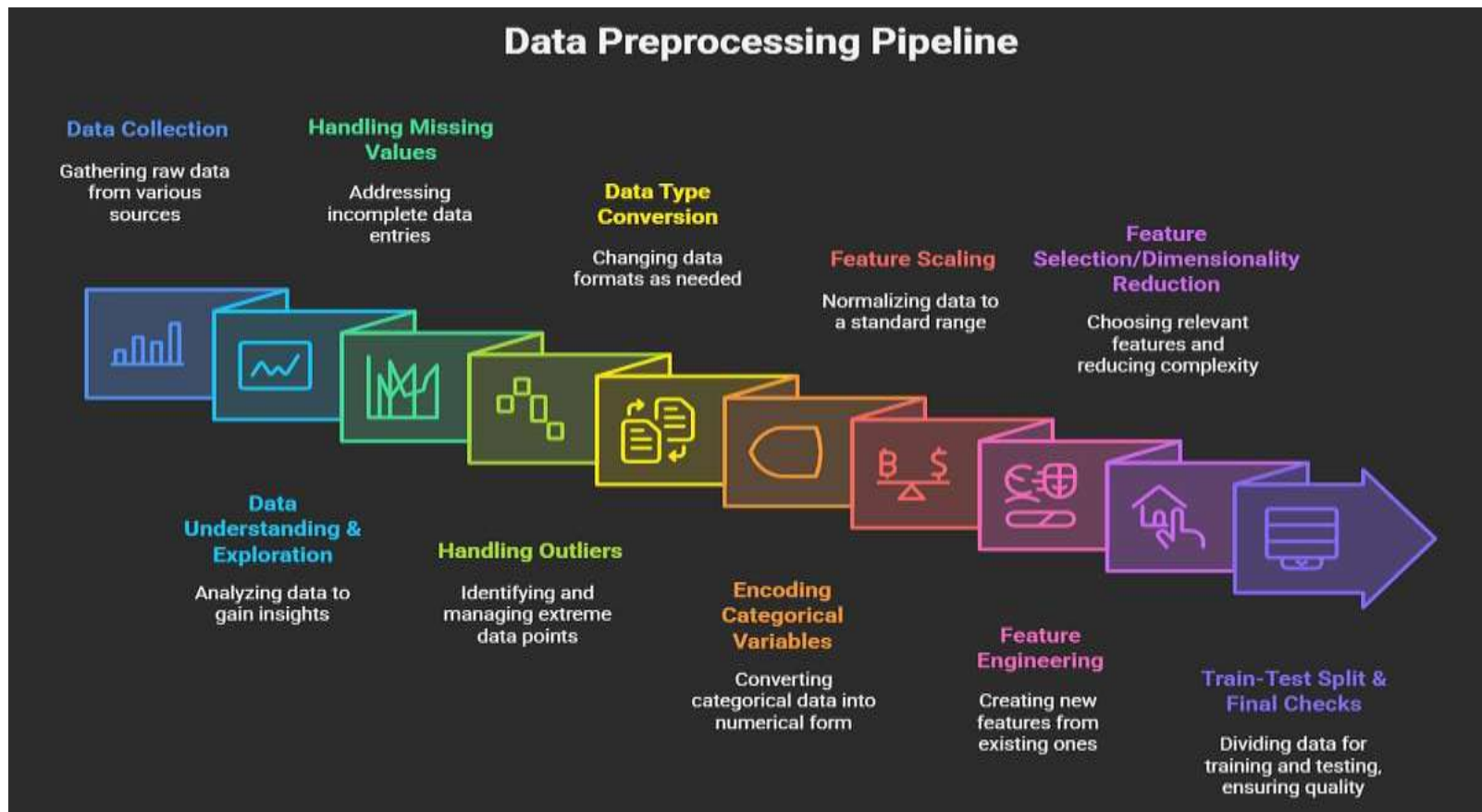
Critical Impact Areas:

- Model Accuracy: Poor data leads to biased/inaccurate predictions
- Training Efficiency: Clean data reduces computational overhead by 40-60%
- Business Outcomes: High-quality insights drive decision-making
- Risk Mitigation: Prevents data leakage and model failures

The Cost of Skipping:

- Model fails in production, ROI loss, reputation damage

workflow



READING CSV FILE USING PANDAS

Reading data means importing datasets into Python for analysis.

Most real-world datasets are stored in CSV (Comma Separated Values) format.

Sample Dataset

Name	Age	Department	Salary
Alice	25	IT	50000
Bob		HR	45000
Charlie	30	IT	55000
David	28	Sales	48000

code:

```
import pandas as pd
df = pd.read_csv("employees.csv")
print(df.head())
```

output

	Name	Age	Department	Salary
0	Alice	25.0	IT	50000
1	Bob	NaN	HR	45000
2	Charlie	30.0	IT	55000

DATA SELECTION & FILTERING

Selecting only the required columns or rows from a dataset.

Selecting Columns

```
df["Salary"]
```

Selecting Multiple Columns

```
df[["Name","Salary"]]
```

Filtering Records

```
df[df["Salary"]>45000]
```

Output

Name	Salary
Alice	50000
Charlie	55000
David	48000

HANDLING MISSING VALUES

Missing values are empty or unknown values represented by NaN (Not a Number).

Methods

Check Missing Values

```
df.isnull().sum()
```

Replace Missing Values

```
df.fillna(df["Age"].mean(), inplace=True)
```

Remove Missing Values

```
df.dropna()
```

Example

Age

25

NaN

30

Age

25

27.5

30

DATA CLEANING

Data cleaning removes errors from datasets.

Why?

Improves accuracy

Removes repeated records

Produces reliable analysis

Remove Duplicate Records

- `df.drop_duplicates(inplace=True)`

Employee ID	Name	Department	Salary
101	Alice	IT	50000
102	Bob	HR	45000
103	Charlie	IT	55000
103	Charlie	IT	55000
104	David	Sales	48000

Employee ID	Name	Department	Salary
101	Alice	IT	50000
102	Bob	HR	45000
103	Charlie	IT	55000
104	David	Sales	48000

SORTING DATA

Sorting arranges data in ascending or descending order.

Why?

- Finds highest and lowest values
- Makes reports easier to understand

Ascending Order

```
df.sort_values("Salary")
```

Descending Order

```
df.sort_values("Salary", ascending=False)
```

Example

Before	After
55000	55000
50000	50000
45000	45000

GROUPING & RANKING

Grouping

Grouping combines similar records together.

Example

Department-wise Average Salary

```
df.groupby("Department")["Salary"].mean()
```

Output

Department Average Salary

HR	46000
IT	52500
Sales	48000

Ranking

Ranking assigns positions based on values.

```
df["Rank"]=df["Salary"].rank(ascending=False)
```

Output

Employee	Salary	Rank
Charlie	55000	1
Alice	50000	2
David	48000	3

FUNDAMENTAL PYTHON LIBRARIES

Library Purpose

NumPy: Numerical Computing

Pandas: Data Manipulation

SciPy: Scientific Computing

Scikit-Learn: Machine Learning

Matplotlib: Visualization

Why These Libraries?

They simplify complex data science tasks such as numerical computation, preprocessing, visualization, and machine learning.

NUMPY, SCIPY & SCIKIT-LEARN

NumPy

Numerical array processing.

```
import numpy as np
```

```
a=np.array([1,2,3])
```

Used for

Arrays

Matrix operations

Mathematical computations

Scikit-Learn

Machine Learning Library.

Used for

Classification

Regression

Clustering

Data Preprocessing

Model Evaluation

SciPy

Scientific Computing Library.

Used for

Statistics

Optimization

Linear Algebra

Signal Processing

DATA VISUALIZATION USING MATPLOTLIB

What is Data Visualization?

Presenting data graphically for easy understanding.

Why?

Detect trends

Compare values

Identify patterns

Improve decision-making

Types of Charts

Bar Chart

Line Chart

Scatter Plot

Histogram

Pie Chart

```
import matplotlib.pyplot as plt
plt.bar(df["Name"], df["Salary"])
plt.xlabel("Employee")
plt.ylabel("Salary")
plt.title("Employee Salary")
plt.show()
plt.bar(df["Name"], df["Salary"])
plt.plot(df["Name"], df["Salary"])
plt.scatter(df["Age"], df["Salary"])
plt.hist(df["Salary"])
plt.pie(department_count,
labels=department_count.index,
autopct="%1.1f%%")
```